

DriftWatch — pipeline scraping yang memantau dirinya sendiri

Studi kasus satu halaman · September 2026

Masalah

Scraper yang “sudah jadi” hampir tidak pernah rusak dengan berisik. Ia rusak diam-diam: satu class CSS berganti nama, satu kolom jadi kosong, satu halaman mulai membalas 503. Datanya tetap mengalir, jumlah barisnya masih masuk akal, dan tidak ada yang curiga — sampai klien menemukan sendiri bahwa laporan tiga minggu terakhir salah.

Kegagalan yang mahal bukan scraper yang mati. Kegagalan yang mahal adalah scraper yang tetap hidup sambil menghasilkan data yang salah.

Pendekatan

Empat keputusan yang membentuk seluruh sistem:

Turun lapis secepatnya. Browser hanya dipakai sekali, untuk recon. Begitu recon menemukan endpoint JSON di balik halaman, browser dibuang. Target quotes: 8 request browser menjadi 1 request httpx untuk data yang sama; panen penuh 100 kutipan = 10 request, nol proses browser, nol dependensi Playwright di mesin klien.

Snapshot tidak pernah ditimpa. Tiap run menulis ke `data/<target>/<tanggal>/` sendiri. Menimpa snapshot kemarin berarti menghapus baseline diff — yaitu menghapus satu-satunya cara mengetahui bahwa hari ini berbeda.

Diff jadi produk utama, bukan efek samping. Tiap record punya `content_hash` atas JSON kanonik, dengan field volatil (`fetch_at`, `run_id`, durasi) sengaja dikecualikan. Tanpa pengecualian itu semua record akan tampak berubah tiap hari dan diff jadi sampah.

Alarm harus dibuktikan patah, bukan diklaim. Sebelas skenario kerusakan ditanam di fixture lokal yang sengaja dimutasi, lalu pipeline produksi yang sama dijalankan di atasnya. Detektor tidak pernah tahu skenario mana yang aktif — persis seperti hari kerja sungguhan.

Hasil

Sepuluh kode alarm dengan ambang tertulis (nol record, jumlah anjlok, kelengkapan turun, field asing, lonjakan error HTTP, run terlewat, run gagal, durasi anomali, pelanggaran rate limit, churn spike). Pipeline mengawasi kesopanannya sendiri: melanggar jeda 1 detik memicu alarm pada run itu juga.

Ia juga menangkap keagalannya sendiri. Selama pengembangan, fixture lokal tidak pernah dinyalakan saat timer memicu — tiga hari berturut menghasilkan nol record dan unit gagal. Alarm berbunyi setiap hari itu; yang belum ada adalah yang membacanya. Perbaikannya masuk ke pipeline, bukan ke catatan.

Angka

Klaim	Angka	Cara dibuktikan
Volume dataset	1.323 record/hari, 4 target, 0 duplikat	<code>wc -l records.jsonl</code>
Kelengkapan field wajib	100,0% di keempat target	<code>run.json field_completeness</code>
Tahan diputus	<code>kill -9</code> lalu <code>--resume</code> : 12 sampai 1.000 record, 0 duplikat	<code>docs/RESUME_PROOF.md</code>
Kesopanan request	jeda minimum teramati 1.000-1.001 ms	<code>run.json rate_limit</code>
Jalan sendiri	3 hari berturut, 12/12 run <code>exit 0</code> , 0 alarm, 0 intervensi	<code>docs/SOAK_PROOF.md</code>
Alarm terbukti patah	11/11 skenario lolos, 0 false positive	<code>make oracles</code>
Reproducible	<code>make all</code> di salinan bersih: <code>exit 0</code> , 18 menit, tanpa Docker	<code>docs/ACCEPTANCE.md A11</code>
Kebocoran rahasia	0	<code>make audit</code>
Biaya ringkasan AI	\$0,00 (opsional, mati secara default)	<code>web/data.json</code>

Seluruh pipeline berjalan dengan Python, SQLite, dan systemd. Tidak ada Docker, tidak ada database jaringan, tidak ada antrian pekerjaan — karena setiap layanan tambahan ikut jadi syarat pemasangan di mesin klien.